

CLINICAL AI CONTROL PLANE

Architecture Portfolio

Clinical AI Control Plane · Complete Documentation Set

Prepared by Engineering · 2026-08-21

Clinical AI Platform Group

PHI PROTECTION · FULL AUDIT TRAIL · HIPAA ALIGNED

Contents

Five documents covering the architecture, its diagrams, its decisions, its standards, and its research

CAG-01 **Technical Architecture**
Layered design, the governed pipeline, state machine, security, API, deployment

CAG-02 **Architecture Diagrams**
System context, governed request flow, state machine, deployment topology

CAG-03 **Architecture Decision Records**
Context, decision, and consequences for every load-bearing choice

CAG-04 **Enterprise Standard**
The layered contract, security posture, code quality bar, anti-pattern table

CAG-05 **Model Provider and Redaction Research**
Model backend options, PHI de-identification options, hallucination posture

CAG-01

Technical Architecture

Governance Gateway for Clinical AI Assistants

Clinical AI Control Plane: Architecture

Version 0.1.0 · Java 21 · Spring Boot 3.3 · PostgreSQL · Redis · Kafka

SECTION 01

Overview

The control plane is a governance gateway that sits between clinical AI assistants (billing, clinical, front office, patient, support) and the model layer. It provides the cross-cutting capabilities an assistant fleet needs in a healthcare setting: **protection of PHI, per-tenant cost and usage observability, configurable human review, provenance for every output, and a complete audit trail**.

The default posture is **autonomy with observability**. Human review and hard budgets are policies an operator turns on where risk justifies them, not fixed gates applied to every request. The model call itself is thin and sits behind a vendor-neutral provider seam; the engineering value is in the governance plane around it.

SECTION 02

Layered design

```
HTTP -> Controller (thin) -> SuggestionService -> ModelProvider seam -> Mock | Re
              JWT: tenant_id,        |                                    (plugga
              scope claims           +-> UsageMeteringService (Redis)
                                     +-> RedactionService (PHI)
                                     +-> GovernancePolicy (review?)
                                     +-> Repository (Postgres, Flyway)
                                     +-> AuditPublisher (Kafka) -> AuditConsumer
```

- **Controllers (thin):** extract `tenant_id` and actor from the verified JWT, validate the DTO, call the service, return the response DTO. No query building, no policy logic.
- **SuggestionService (single orchestrator):** owns the governed lifecycle. Every request passes through the same plane in the same order.
- **Provider seam:** `ModelProvider` is the only interface the model is called through. `MockModelProvider` is the default; `RemoteModelProvider` is the marked seam for a real client.
- **Audit is a separate consumer:** transitions are published to Kafka and persisted to a Postgres read model by `AuditConsumer`, decoupled from the write path.

SECTION 03

The governed pipeline

1. An assistant calls the gateway with a JWT carrying a `tenant_id` claim and scopes.
2. `UsageMeteringService.admit()` meters the request per tenant against a Redis counter. Monitor mode (default) records and warns without blocking; enforce mode throws `RateLimitExceededException` over budget.
3. `RedactionService` strips direct identifiers from the input **before any model call**. Redaction is always on; it is a data-protection concern, not a policy toggle.
4. The model is called through the provider seam with the redacted text. Token usage is recorded back to the meter.
5. The `Suggestion` is persisted with its grounding references and provenance (provider, tokens, actor).
6. `GovernancePolicy` decides the next state: auto-approve as actor `policy:auto` unless the assistant type appears in `REVIEW_REQUIRED_ASSISTANTS`, in which case the suggestion stays `PENDING` for an authorized reviewer.
7. Every transition, automated or human, publishes an `AuditEvent` to Kafka.

SECTION 04

Suggestion state machine

```
PENDING --approve(actor)--> APPROVED --markExecuted()--> EXECUTED
   |
   +----reject(actor, reason)--> REJECTED
```

Transitions are enforced inside the `Suggestion` entity: decisions are only legal from `PENDING`, and only an `APPROVED` suggestion can be executed. Invalid transitions throw `IllegalStateException` rather than silently correcting. The policy actor (`policy:auto`) and human actors travel the same transitions, so the audit trail is uniform across autonomous and reviewed paths.

SECTION 05

Multi-tenancy and security

- Stateless OAuth2 resource server; every `/api` request requires a JWT verified against the configured public key. Only actuator health/info, the OpenAPI UI, and `/dev/**` are open.
- **Tenant isolation at the query:** the `tenant_id` claim is mandatory, and every repository read filters by it (`findByIdAndTenantId`). A tenant can never address another tenant's suggestion id.
- **Scope-gated operations:** `suggestion:create`, `suggestion:approve` (approve and reject), and `suggestion:execute` are independent authorities checked with method-level `@PreAuthorize`.
- The `dev` profile enables a local token minting endpoint (`GET /dev/token`) signed by a throwaway RSA key pair used only for local development and tests. Production validates tokens from a real identity provider.

SECTION 06

Audit trail

Every lifecycle event (`SUGGESTION_CREATED`, `SUGGESTION_AUTO_APPROVED`, `SUGGESTION_APPROVED`, `SUGGESTION_REJECTED`, `SUGGESTION_EXECUTED`) is published to the `clinical-ai.audit` topic and persisted by a separate consumer into a Postgres read model, queryable per tenant and per suggestion.

Publishing currently happens inside the write transaction for clarity. The dual-write between the database and Kafka is a known seam; the production-grade answer is a **transactional outbox**, recorded in ADR 0003.

SECTION 07

API contract

ENDPOINT	SCOPE	PURPOSE
`POST /api/v1/suggestions`	`suggestion:create`	Run the governed pipeline; returns the suggestion with status and provenance.
`GET /api/v1/suggestions`, `GET /api/v1/suggestions/{id}`	authenticated	List and read, always tenant-filtered.
`POST /api/v1/suggestions/{id}/approve`, `/reject`	`suggestion:approve`	Human decision from `PENDING`.
`POST /api/v1/suggestions/{id}/execute`	`suggestion:execute`	Mark an approved suggestion executed downstream.
`GET /api/v1/usage`	authenticated	Per-tenant usage counters for cost observability.
`GET /api/v1/audit`, `GET /api/v1/suggestions/{id}/audit`	authenticated	Audit read model, per tenant and per suggestion.

The contract is described by springdoc OpenAPI and served at `/swagger-ui.html`. Errors return a structured `ApiError` body via the global exception handler.

SECTION 08

Configuration surface

VARIABLE	DEFAULT	PURPOSE
`MODEL_PROVIDER`	`mock`	Model backend behind the seam: `mock` for local and tests, `remote` for a real backend.
`USAGE_MODE`	`monitor`	`monitor` records without blocking; `enforce` applies the hard budget.
`USAGE_RPM`	`600`	Per-tenant request budget for the soft warning and enforce mode.
`REVIEW_REQUIRED_ASSISTANTS`	empty	Comma-separated assistant types requiring human review. Empty means autonomy.

SECTION 09

Deployment topology

- **Container:** multi-stage Docker build (`docker/Dockerfile`); CI builds the image on every push.
- **One chart, two targets:** the Helm chart under `helm/clinical-ai-gateway` deploys both the local minikube proof (`deploy/local/up.sh` , pinned to the `minikube` context) and the EKS production target.
- **Infrastructure as code:** Terraform provisions the VPC and EKS cluster (`terraform/`).
- **CI gates:** build and Testcontainers-backed tests, Spotless formatting as a separate job, Trivy scan (vuln, secret, misconfig), and a Docker image build.

SECTION 10

Testing strategy

- **Unit:** redaction patterns, metering in monitor and enforce modes, and orchestration across the autonomous and review-required paths (Mockito).
- **Integration:** Testcontainers starts Postgres, Kafka, and Redis; the test drives the create-and-approve flow over HTTP with a signed token and awaits the audit consumer persisting the trail (Awaitility).

SECTION 11

Scope, dependencies, and assumptions

- The model call is mocked by default; the remote provider is a stub marking where a real client belongs.
- Redaction is pattern-based and illustrative; a production deployment would use a clinical NLP de-identification service behind the same boundary.
- PHI redaction and the audit trail are **always on**; they are data-protection and compliance concerns, not limits on what the AI can do.
- Schema changes go through Flyway migrations only (`ddl-auto: none`).
- Design rationale is recorded in ADRs 0001 through 0006 (companion document CAG-03).

CAG-02

Architecture Diagrams

System Context and Request Flow

Clinical AI Control Plane: Architecture Diagrams

System context · Governed request flow · State machine · Deployment topology

SECTION 01

System context

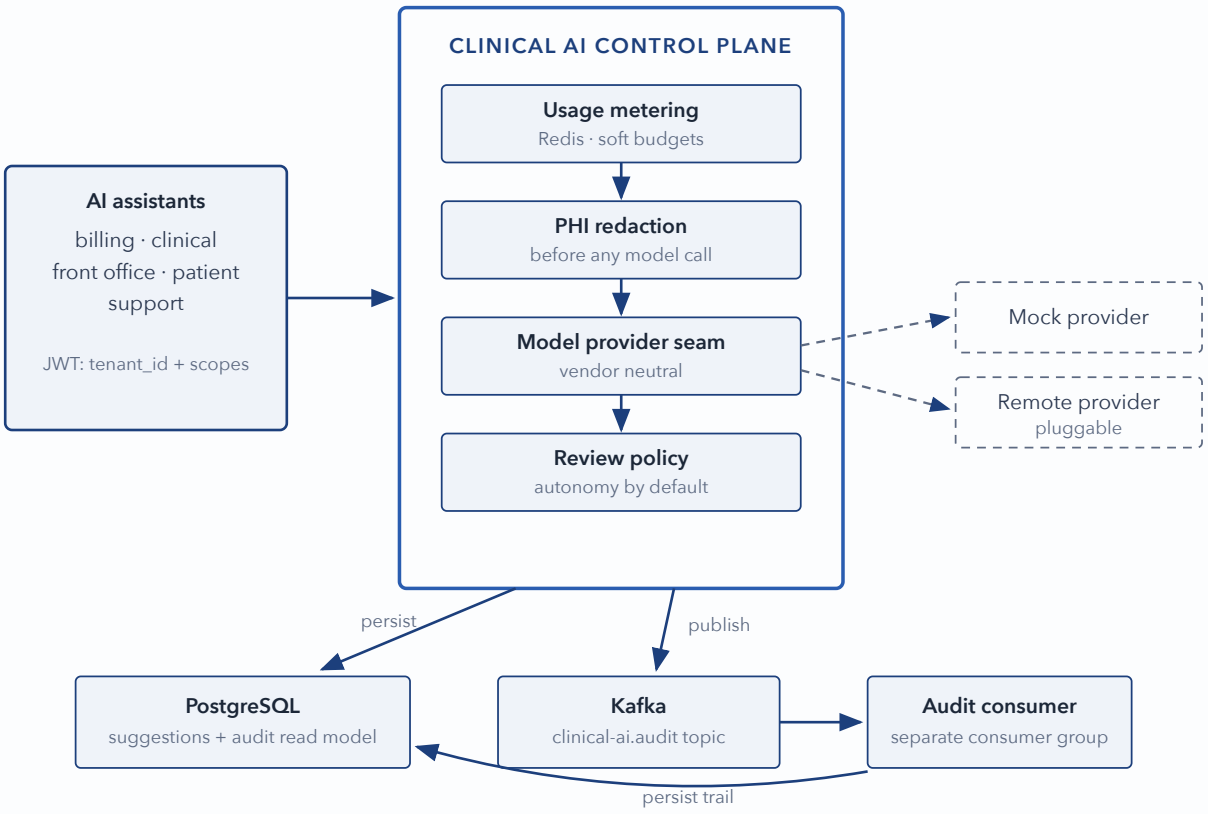


Figure 1 · The control plane between the assistant fleet and the model layer

SECTION 02

Governed request flow

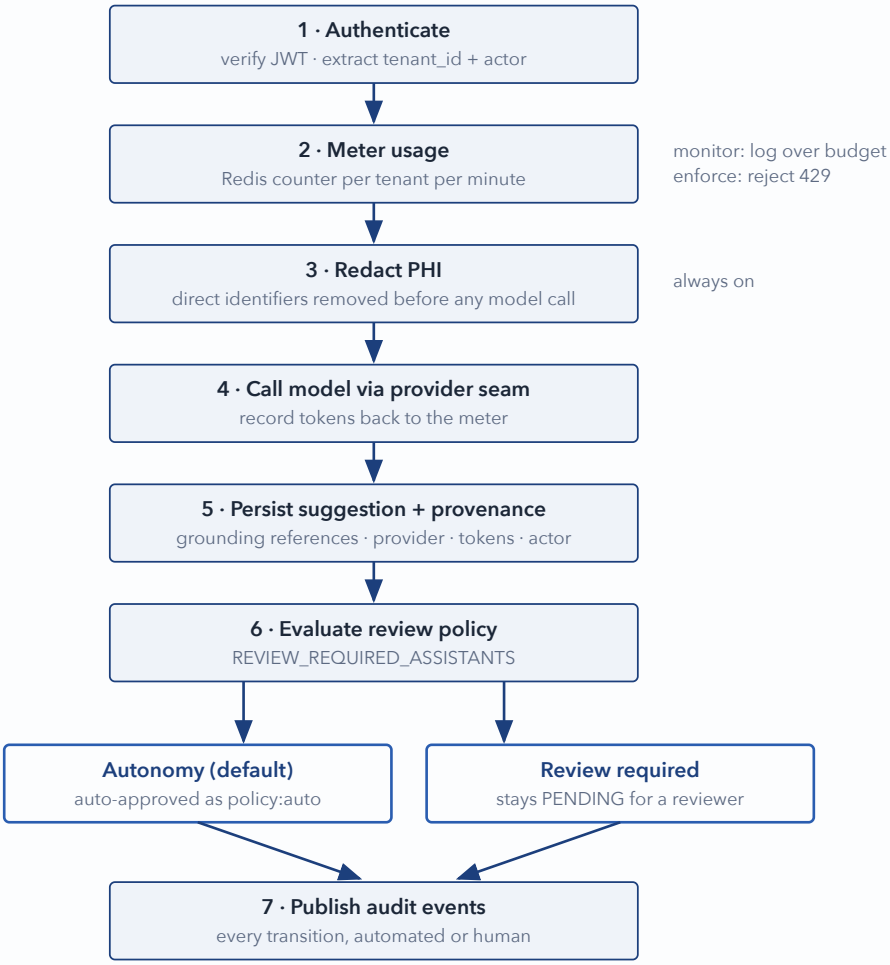


Figure 2 · Every request passes through the same plane in the same order

SECTION 03

Suggestion state machine

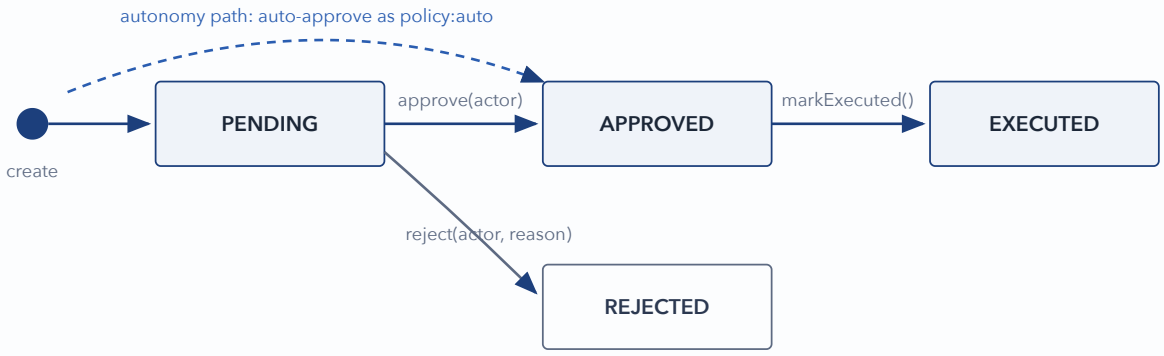


Figure 3 · Transitions enforced in the entity; decisions only from PENDING, execution only from APPROVED

SECTION 04

Deployment topology

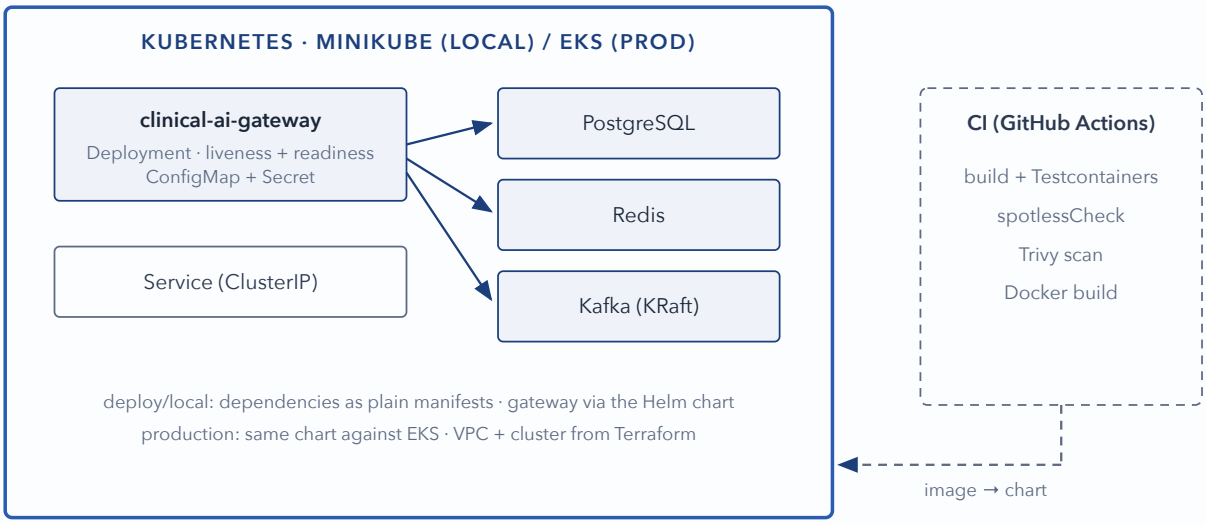


Figure 4 · One Helm chart serves the local minikube proof and the EKS target

SECTION 05

Decisions these diagrams encode

- Assistants never call the model directly; every request crosses the control plane, so governance is defined once and applied uniformly (ADR 0001).
- Review is a policy outcome inside the flow, not a separate approval system; the autonomous and reviewed paths share one state machine and one audit trail (ADR 0002).
- The audit trail leaves the request path through Kafka and lands in a separate read model, so new consumers attach without touching the producer (ADR 0003).
- PHI redaction sits at the gateway boundary, upstream of the provider seam, so no provider contract determines what data leaves the trust boundary (ADR 0004).
- The provider seam keeps the model layer swappable; the mock provider makes the whole plane runnable and testable with no cloud dependency (ADR 0005).
- Metering is observability first; enforcement is an opt-in mode, so a cost signal never becomes an availability incident by default (ADR 0006).
- One Helm chart deploys both targets, so the local cluster is a faithful rehearsal of production rather than a separate configuration.

END OF DOCUMENT · CAG-02

CAG-03

Architecture Decision Records

Six Accepted Decisions

Clinical AI Control Plane: Architecture Decision Records

Six accepted decisions · each with context, decision, and consequences

SECTION 01

What the control plane builds

A governance gateway between a fleet of clinical AI assistants and the model layer: PHI redaction, per-tenant usage metering, policy-driven human review, provenance, and a Kafka-backed audit trail, exposed as a versioned REST API (`/api/v1`) secured as an OAuth2 resource server.

Decided and closed: governance is centralized in one plane rather than re-implemented per assistant. Per-assistant controls were rejected: controls drift apart, every new assistant repeats the work, and a compliance reviewer has no single place to verify behavior.

ADR 0001 · ACCEPTED

Centralized AI control plane

Context. The product direction is a fleet of assistants spanning billing, clinical, front office, patient, and support workflows. Each needs the same cross-cutting controls: PHI protection, per-tenant cost and abuse limits, human review for actions affecting a patient record, provenance, and a complete audit trail.

Decision. Assistants do not call the model directly. They call the control plane, which enforces redaction, metering, the review policy, provenance capture, and audit publishing, then delegates the model call through a provider seam. The plane owns the governed lifecycle of a suggestion from creation to a terminal decision.

Consequences. Governance is defined once and applied uniformly; a new assistant inherits the controls for free and a reviewer has a single system of record. The plane becomes a shared dependency on the request path, so it stays stateless at the web tier, backs state in Postgres, and keeps the model call behind an interface so it can be made asynchronous if the latency budget requires it.

ADR 0002 · ACCEPTED

Policy-driven human review

Context. Suggestions range from low-impact drafts to clinical and financial actions. Holding every output for approval slows low-impact paths without adding safety; reviewing nothing is unsafe for high-impact paths. Oversight should match the risk of the action.

Decision. Review is a policy decision, not a hard gate. The plane evaluates `app.governance.review-required-assistants` at creation time. Empty by default, so the default posture is autonomy; a risk-based list such as `CLINICAL,BILLING` turns on review for exactly those flows. The state machine and audit trail apply on every path, including auto-approval.

Consequences. No friction where it adds no value; human oversight exactly where risk justifies it, without code change. Every decision records an actor. The tradeoff: the policy itself becomes a governed artifact that must be reviewed and tested. It currently keys on assistant type and can extend to action risk, model confidence, or tenant configuration behind the same interface.

ADR 0003 · ACCEPTED

Kafka for the audit trail

Context. Every state change must produce a durable audit record for frameworks such as HIPAA and SOC 2. The trail must outlive the request, be queryable, and serve multiple consumers over time (compliance store, analytics, alerting). Direct Postgres writes in the same transaction and RabbitMQ were considered.

Decision. State changes publish an audit event to a Kafka topic keyed by suggestion id, so one suggestion's history stays ordered within a partition. A dedicated consumer persists events into a Postgres audit read model. Kafka retains the event log and lets independent consumer groups read the same stream without coupling.

Consequences. The trail is decoupled from the request transaction and fans out to new consumers without producer changes. The cost is an eventual-consistency window and a dual write between the database and Kafka. The production answer is a **transactional outbox**: the event is written to an outbox table in the same transaction and relayed to Kafka. The current code publishes inline behind a single component, so the move to an outbox is a contained change.

ADR 0004 · ACCEPTED

PHI redaction at the gateway boundary

Context. Assistant inputs can contain protected health information. Model providers may run outside the trust boundary, and even a provider under a business associate agreement should receive the minimum necessary data. A consistent removal point is needed rather than trusting each assistant or provider contract.

Decision. The plane redacts direct identifiers from inbound text at the gateway boundary, before the provider is called, for every assistant, through a single service behind a stable interface. The current implementation is pattern-based and illustrative; production replaces the implementation with a dedicated clinical detection service behind the same interface, without changing call sites.

Consequences. Redaction is applied once, uniformly, and independently of any provider contract, giving one place to test and improve detection. Tradeoffs: over-redaction can reduce model quality, and pattern-based detection is not sufficient alone for free-text clinical notes. Relying on a provider agreement alone was rejected: a contract does not reduce the data sent.

ADR 0005 · ACCEPTED

Vendor-neutral model provider seam

Context. The backend will change over time and must not be hardcoded into governance logic. The platform must not couple to a single vendor, needs a deterministic backend for local development and tests, and wants the option to route or fall back across backends.

Decision. Model access sits behind a `ModelProvider` interface; governance code never references a concrete backend. A mock provider returns deterministic, specialty-shaped output and is the default. A remote provider, selected by configuration, is the seam where a real backend is wired: an internal model gateway, a hosted model API, or a managed cloud service. Routing or fallback can be layered in front later.

Consequences. The backend can be swapped or composed without touching redaction, metering, policy, or audit. Tests run end to end with no external dependency and no model cost. Vendor choice is deferred to deployment configuration. Tradeoffs: one layer of indirection and a result contract that must stay stable across providers so token accounting stays uniform. A hardcoded provider was rejected.

ADR 0006 · ACCEPTED

Usage metering and soft budgets, not hard rate limits

Context. The plane needs visibility into cost and abuse: model calls carry real spend and a single tenant can drive large volume. The obvious approach, a hard rate limit, blocks a clinician or a patient-facing assistant mid-workflow and turns a cost concern into a reliability problem.

Decision. The default posture is metering, not throttling. Per-tenant request counts and token usage are recorded in Redis; crossing a soft budget is logged without blocking. A hard enforcement mode exists behind `app.usage.mode=enforce` for tenants or environments that explicitly opt into a cap. Enforcement is off by default.

Consequences. Assistants stay responsive by default and the cost signal feeds dashboards and alerts rather than outages. Operators can enable a hard ceiling per environment without code change. Tradeoff: metering alone does not stop a runaway tenant's in-the-moment spend, so it must pair with alerting and, where required, opt-in enforcement. The token counters are the foundation for per-tenant cost reporting and a future token budget.

SECTION 08

Acceptance criteria

- Every suggestion, on every path, has a complete audit trail with an actor for each transition, verified end to end by the Testcontainers integration test.
- No model call ever receives unredacted input; redaction is exercised by unit tests and cannot be disabled by configuration.
- Review policy changes require configuration only, never a code change, and are covered by tests on both the autonomous and review-required paths.
- The whole plane runs locally with no cloud dependency: Docker Compose for infrastructure, mock provider for the model, minikube for the cluster proof.

SECTION 09

Open items

- [ ] Replace the inline audit publish with the transactional outbox described in ADR 0003.
- [ ] Wire a production de-identification service behind `RedactionService` (candidates evaluated in CAG-05).
- [ ] Implement the remote provider against the chosen backend (options evaluated in CAG-05).
- [ ] Attach alerting to the soft-budget log signal so metering closes the loop operationally.
- [ ] Extend `GovernancePolicy` beyond assistant type: action risk, model confidence, tenant configuration.

END OF DOCUMENT · CAG-03

CAG-04

Enterprise Standard

Coding and Architecture Standard

Clinical AI Control Plane: Enterprise Coding and Architecture Standard

The coding contract the implementation conforms to and the reference the architecture spec builds on

SECTION 01

Layered infrastructure (strict, one direction of dependency)

```
HTTP -> Controller (thin) -> Service (orchestration) -> Repository -> PostgreSQL
             input DTO / validator        |                (tenant-filtered reads
             <- typed response DTO <-     +-> ModelProvider | RedactionService |
                                              UsageMeteringService | GovernancePo
                                              AuditPublisher
```

- **Controller (thin):** parse the request into a validated DTO, extract tenant and actor from the verified JWT, call the service, return the response DTO. No query building, no persistence, no inline policy logic.
- **Service (single orchestrator):** `SuggestionService` owns the governed lifecycle: meter, redact, call the provider, persist, evaluate policy, audit. Domain rules that belong to the entity stay in the entity.
- **Repository:** Spring Data interfaces only. Every read that returns tenant data takes the tenant id in the query (`findByIdAndTenantId` , `findByTenantIdOrderByCreatedAtDesc`). No post-load filtering.
- **DTO boundary:** entities never cross the HTTP boundary. Request records (`CreateSuggestionRequest` , `DecisionRequest`) validate with Jakarta annotations; response records (`SuggestionResponse` , `AuditEventResponse`) shape the contract.

SECTION 02

Configuration over code

- Every governance behavior is externalized under a typed `app.*` namespace in `application.yml` with environment-variable bindings and safe defaults: `app.model.provider`, `app.usage.mode`, `app.usage.requests-per-minute`, `app.governance.review-required-assistants`, `app.kafka.audit-topic`.
- Changing where autonomy applies, which model backend runs, or whether budgets enforce is a deployment change, never a code change.
- No magic strings on the configuration path; domain vocabulary is typed (`AssistantType`, `SuggestionStatus`, `GroundingSourceType`, `AuditEventType` enums). An invalid assistant type fails validation at the boundary, not silently downstream.

SECTION 03

Security first (non-negotiable in this compliance environment)

- **Stateless resource server:** every `/api` request carries a verified JWT; sessions and CSRF are disabled because there is no state to protect. Only health probes, the OpenAPI UI, and the dev token endpoint are open.
- **Tenant scope in the query, resolved once:** the `tenant_id` claim is mandatory and applied at the repository query. A missing claim is an error, never a silent fallback.
- **Scope-gated writes:** create, decide (approve/reject), and execute are separate authorities enforced with method-level `@PreAuthorize`, so a reviewer credential cannot create and a creator credential cannot approve.
- **Data minimization before the model:** PHI redaction runs before any provider call, always on, not configurable off. Provider contracts never determine what leaves the trust boundary.
- **Read and write audit:** every lifecycle transition emits an audit event with tenant, actor, and detail; the trail is persisted by an independent consumer and queryable per tenant and per suggestion.
- **Structured errors, never silent skips:** invalid transitions throw, unknown ids return a typed 404, budget rejections return a typed 429; all through one `GlobalExceptionHandler` returning a structured `ApiError`.
- **Secrets posture:** the only key in the repository is a throwaway dev signing key, documented as such; production keys come from the identity provider and a managed secret store. CI runs a Trivy scan for vulnerabilities, secrets, and misconfigurations.

SECTION 04

Enterprise code quality

- **Typed everything:** records for DTOs and value objects, enums for states and types, interfaces for swappability (`ModelProvider`). The state machine lives in the `Suggestion` entity and throws on invalid transitions.
- **Schema by migration only:** Flyway owns the schema (`V1__init.sql`); Hibernate runs with `ddl-auto: none` . A schema change is a new versioned migration, reviewed like code.
- **First-party, supported components** on the request path: Spring Web, Spring Security, Spring Data JPA, Spring Kafka, Spring Data Redis. No experimental drivers.
- **Formatting as a gate:** google-java-format via Spotless; `spotlessCheck` runs as its own CI job so a fresh clone builds green, and `spotlessApply` runs before commit.
- **Tests at two altitudes:** unit tests (Mockito) for redaction, metering in both modes, and orchestration on both policy paths; one integration test (Testcontainers: Postgres, Kafka, Redis) driving the real HTTP flow with a signed token and awaiting the audit consumer.
- **Contract first:** the API is described by springdoc OpenAPI and served from the running application, so documentation cannot drift from the implementation.
- **Observability:** actuator health probes wired to Kubernetes liveness and readiness; per-tenant usage counters exposed over the API as the cost signal.

SECTION 05

Anti-pattern replacements

COMMON DEFECT	STANDARD IN THIS CODEBASE
Assistants call the model vendor directly	All model access through the control plane's `ModelProvider` seam
PHI handling delegated to provider contracts	Redaction at the gateway boundary, always on, before any call
Review rules hardcoded per feature	`GovernancePolicy` driven by configuration; autonomy by default
Hard rate limits as the first cost control	Metering and soft budgets; enforcement is an explicit opt-in mode
Audit as scattered log statements	Typed `AuditEvent` stream on Kafka with a persistent read model
Status changed by setters from anywhere	State machine methods on the entity; invalid transitions throw
Tenant filtering in application code after load	Tenant id bound into every repository query
Hibernate auto-DDL mutating production schema	Flyway migrations only, `ddl-auto: none`
Entities serialized straight to JSON	Request and response DTOs at the HTTP boundary
Formatting debates in review	google-java-format enforced by a CI gate

This standard is the coding contract for the control plane implementation and the reference the architecture specification (CAG-01) builds on.

CAG-05

Model Provider and Redaction Research

Production Seam Feasibility

Clinical AI Control Plane: Model Provider and PHI Redaction Research

Feasibility of the production seams · provider options · de-identification options · local dev

SECTION 00

Headline finding (read this first)

The governance plane is provider-proof by construction. Every candidate model backend and every candidate de-identification service plugs in behind an existing interface (`ModelProvider` , `RedactionService`) without touching metering, policy, persistence, or audit. Vendor selection is therefore a deployment decision with a contained blast radius, not an architecture decision. The two real constraints found are: the `ModelResult` contract (provider, output, tokens) must stay uniform across backends so token accounting and cost reporting do not fork, and redaction quality, not redaction placement, is the open engineering problem for free-text clinical notes.

SECTION 01

How the provider seam constrains the pipeline

- **The seam is called after redaction and inside metering.** Any backend receives only redacted text, and its token usage is recorded to the per-tenant meter. A provider integration must report token counts; a backend that cannot report usage breaks cost observability and needs a token-estimation shim.
- **The result contract is the invariant:** `ModelResult(provider, output, tokensUsed)` . Provenance stores the provider string per suggestion, so responses must be attributable to a specific backend and model version for the audit trail to stay meaningful. Recommendation: extend `provider` to carry model id and version when the remote provider is wired.
- **Synchronous today, async-ready:** the model call sits on the request path. Latency budgets for real backends (hundreds of ms to seconds) may require the call to become asynchronous; ADR 0001 anticipated this, and the seam means the change does not disturb governance.
- **Routing and fallback compose in front of the seam:** a routing provider that wraps several concrete providers satisfies the same interface, which is where model tiering (small model for drafts, large model for clinical text) belongs.

SECTION 02

Model backend options

OPTION	WHAT IT IS	STRENGTHS	CONSIDERATIONS	FIT
AWS Bedrock	Managed multi-model API (Claude and others) inside the AWS account	HIPAA-eligible service; traffic stays in the AWS boundary; IAM-native auth; fits the existing EKS/Terraform footprint	Model availability and pricing vary by region; per-model invocation quotas need capacity planning	**Best fit for the current AWS deployment target**
Anthropic Claude API (direct)	First-party hosted API	Latest models first; strong clinical reasoning; token usage returned per call, mapping cleanly to `ModelResult`	Requires a business associate agreement for PHI-adjacent use; traffic leaves the AWS boundary	Strong fit where the newest model capability matters most
Self-hosted open model (vLLM on EKS)	Open-weights model served in-cluster	Data never leaves the cluster; no per-token vendor cost; full version pinning	GPU node groups, serving expertise, and evaluation burden shift in-house; capability gap for clinical text	Fallback or data-residency option, not the first integration
Mock provider (current default)	Deterministic, specialty-shaped output in-process	Zero cost, zero latency, deterministic tests, no cloud dependency	Not a product capability; exists to prove the plane	Remains the default for local dev and CI permanently

Recommendation. Wire `RemoteModelProvider` against Bedrock first: it keeps PHI-adjacent traffic inside the existing AWS trust and compliance boundary and requires no new credential surface beyond IAM. Keep the direct Anthropic API as a second concrete provider behind the same seam, selected per environment, and revisit routing once two real providers exist.

SECTION 03

PHI de-identification options

The current implementation is pattern-based (SSN, phone, email styles of direct identifiers) and documented as illustrative. ADR 0004 fixes the placement (gateway boundary, always on) and leaves the detection engine swappable. Candidates:

OPTION	WHAT IT IS	STRENGTHS	CONSIDERATIONS	FIT
AWS Comprehend Medical	Managed clinical NLP with a PHI detection API	Purpose-built for clinical text; HIPAA-eligible; managed, no model to maintain; entity types map to HIPAA identifiers	Per-request cost on every gateway call; adds a network hop to the hot path; English-centric	**Best fit for production on the AWS target**
Microsoft Presidio	Open-source PII/PHI detection and anonymization framework	Runs in-cluster, no per-call cost, extensible recognizers, deterministic and testable	Clinical recall depends on the NLP models configured; tuning and evaluation are owned in-house	Strong fit for cost control and data residency; good staging step
Commercial clinical NLP (e.g. John Snow Labs)	Licensed clinical de-identification pipelines	State-of-the-art published accuracy on clinical de-identification benchmarks	License cost; heavier runtime footprint; procurement	Evaluate if measured recall of the first choice is insufficient
Pattern-based (current)	Regex rules for direct identifiers	Zero dependency, fast, fully deterministic for tests	Not sufficient for free-text clinical notes; no recall guarantees	Remains the test/dev engine and the documented seam marker

Recommendation. Comprehend Medical behind `RedactionService` for production, with the pattern engine retained as the deterministic test double. Define a measured acceptance bar before switching: a labeled evaluation set of representative notes, recall on HIPAA identifier categories, and an explicit over-redaction budget, since over-redaction degrades model output quality (the tradeoff ADR 0004 records).

SECTION 04

Hallucination posture: provenance and review as mitigations

- **Grounding references are the anchor.** Every suggestion carries typed references to the source records (encounter, note) it was generated from. A suggestion that cannot cite grounding is reviewable on that basis alone, and downstream validators can check output claims against the referenced sources.
- **The review policy is the risk valve.** Where hallucination cost is high (clinical, billing), `REVIEW_REQUIRED_ASSISTANTS` routes exactly those flows to a human without slowing the rest of the fleet.
- **The audit trail makes model behavior measurable.** Because every suggestion stores provider, output, tokens, and decision outcome, rejection rates per assistant type and per provider are queryable, which is the raw material for an evaluation loop (flagging providers or prompts whose rejection rate drifts).
- **Next step, not yet built:** an automated validation stage between the provider call and persistence (schema checks on structured outputs, grounding-consistency checks) behind its own seam, so validation evolves without touching orchestration.

SECTION 05

Local development workflow

- The mock provider keeps the entire governed flow runnable and testable with **no cloud account, no API key, and no model cost**: Docker Compose supplies Postgres, Redis, and Kafka; the dev profile mints local tokens.
- The integration test proves the plane end to end (HTTP in, audit trail out) with Testcontainers; it will not change when a real provider is wired, because it exercises governance, not the vendor.
- When the remote provider lands, contract tests against recorded provider responses keep CI deterministic; live-provider smoke tests run in a separate, credentialed pipeline stage, never on the default build.

SECTION 06

Open items to close

- [ ] Confirm business associate agreement coverage for the chosen backend (Bedrock service terms, or a direct Anthropic BAA).
- [ ] Extend `ModelResult.provider` to carry model id and version for provenance before the first real integration.
- [ ] Build the labeled de-identification evaluation set and set the recall and over-redaction acceptance bar.
- [ ] Benchmark request-path latency with a real provider; decide whether the model call moves off the synchronous path.
- [ ] Design the validation stage (schema and grounding-consistency checks) as its own seam.